Hoshitori

RYO-CHAN Virtual Hackathon: project submission form. Generated from `docs/submission/form.html` by `scripts/make_submission_form.py`.

Members

Total members	1
Name, role, email	Prateek Tripathi, builder (design, engineering, research). Contact as given in the in-app submission.

Overview

Project	Hoshitori: agents forecast crypto markets in public; every call is time-stamped by two independent authorities before its outcome is known, settled on real exchange prices, and recorded so that anyone can check it without trusting the people who made it.
Track	Track 1, Autonomous Agents; Track 2, Dashboards and Interfaces; Track 3, New Skills. Each is judgeable on its own (README, "Hackathon tracks").
Problem	Market calls, by people and by AI agents, arrive as a screenshot or a confident paragraph. A reader cannot tell when a call was made, whether it was edited afterwards, or whether the losing calls were quietly dropped, so trust goes to whoever sounds most certain.
Solution	A forward-only league. Each cycle the agents read the same frozen snapshot of real exchange data (and RYO's tools when enabled), state a chance or a target-or-stop call, and the whole batch is signed into a hash-chained log and time-stamped by two independent RFC 3161 authorities before the forecast window opens. Calls are settled on real bars from two exchanges. Every cycle exports a bundle that two independent verifiers (Python and Node) and the browser check offline. The interface says what changed, ranks it, shows each call's evidence and what would prove it wrong, and states what has not been shown.

Key features	• A rules agent and three model agents (a single judgment, a bull, bear and judge debate, and a same-budget control) on one snapshot, one clock and one settlement. • The Desk: an investigation agent that picks each check from a menu, tries to disprove its own call, and passes it through gates that can only shrink it. • Independent time-stamps, a signed log, offline verification in two languages and in the browser, a forged-record demonstration. • Failure drills (RYO down, rate limited, exchanges down, model down, time-stamp authority unreachable, a crash mid-run) with the recorded behaviour of each. • A claim reader that cuts a claim into typed parts tied to its exact words, without saying whether it is true. • Seven standalone skills: address flags (OFAC SDN list, USDT and USDC issuer freezes), indicator conditions on the last closed candle, evidence sensitivity, research integrity, exit cost, evidence delta and a scorecard of RYO's own verdicts.
Target users	A crypto trader or researcher who is shown a market call, by a person or an agent, and must decide whether to rely on it; and the builders of RYO-style agents, who can adopt the skills and the verification format.
Scope	Practice only: nothing buys, sells or moves an asset. RYO's six read-only tools are used through the published builder interface; the model is brought by the operator (an OpenAI-compatible endpoint; NVIDIA's in the recorded runs).
Limitations	• Forecast skill is not measured: too few calls have settled for any agent to be ranked. • The impact ranking on Today has not been compared with independent expert labels. • No independent human review, comprehension study, screen-reader study or Japanese-reader study has been run. • RYO's monthly call allowance limits how often RYO evidence can enter a cycle. • `docs/STATUS.md` lists every unit with its evidence and its limit.

Tech stack

Frontend	React 19, TypeScript, Vite; a static site; `@noble/hashes` and `@noble/curves` for in-browser verification.
Backend	Python 3.12 (standard library plus `cryptography` and `asn1crypto`); an append-only, hash-chained ledger with signed checkpoints; RFC 3161 time-stamps; a second verifier and a second settlement implementation in Node.
AI model(s)	Any OpenAI-compatible endpoint chosen by the operator; recorded runs used `nvidia/nemotron-3.5-super-vl-120b-a12b`. The rules agent uses no model.

Other technologies	RYO MCP builder tools (REST); Binance, Kraken and Coinbase public market data; DigiCert and GlobalSign time-stamp authorities; pytest, Hypothesis and mutation sweeps; Playwright and axe-core page tests; GitHub Actions CI.

Repository and demo

GitHub repository	`https://github.com/RYO-Digital/ryochan-hackathon_repository-388` (branch `main`)
Judges' brief	https://congruous-neighbor-289.notion.site/Hoshitori-Brief-for-Users-Judges-Research-PMF-414dfbac05cc8127bad1ddc2dfa28f1e (the product, each track and each rubric line)
Live demo	https://hoshitori-demo.vercel.app (a static export of the record)
Demo video	https://youtu.be/jHUVHBHvaws
Documentation	`README.md` (overview, run, verify, tracks, tests); `docs/STATUS.md` (every unit, its evidence and its limit); `docs/specs/` (specifications); each skill's own README under `skills/`.

Testing information

How to run	Create the environment, run the quickstart (about seven minutes; it reads public market data and asks two public time-stamp authorities for a token), build the interface and serve it. No key and no account are needed.
Prerequisites	Python 3.12 (through `uv` or a plain venv), Node 22.12 or later, network access to Binance, Kraken, DigiCert and GlobalSign.
Installation	`uv venv --python 3.12 .venv && uv pip install -e ".[dev]" ./skills/indicator-condition-check`
Environment variables	None for the quickstart. `.env.example` lists the optional ones: `RYO_MCP_KEY` (RYO tools), `LLM_BASE_URL` and `LLM_API_KEY` (a model), `X_BEARER_TOKEN` (public posts for the Voices page).
Build	`cd web && npm ci && npm run build`
Run	`.venv/bin/python scripts/quickstart.py ../hoshitori-work`, then copy `../hoshitori-work/site` to `web/dist/data` and serve `web/dist` (for example `python3 -m http.server -d web/dist 8000`).
Test	`.venv/bin/python -m pytest -k "not every_change"`; `(cd verifier-js && node --test test/*.test.mjs)`; `(cd settlement-js && node --test test/*.test.mjs)`; `(cd web && npm test)`.
Test accounts	None needed: there is no login. The skills install and answer without a key.